

Headless WordPress Setup Guide

Hostinger + WordPress + WPGraphQL for an editable Next.js portfolio

Prepared for: abdul.qarigroup.com · CMS: cms.qarigroup.com · June 2026

Hostinger Shared/Cloud

WordPress

WPGraphQL

ACF (Free)

Custom Post Type UI

→ Next.js on Vercel

How this fits together

WordPress is your **editable data source only** — no public theme, no visitors. Next.js (on Vercel) fetches your content through the GraphQL API at build time, generates static pages, and rebuilds automatically when you edit content. This keeps the site fast and Hostinger barely loaded.

Layer	Lives on	Domain	Role
Frontend (Next.js)	Vercel (free)	abdul.qarigroup.com	What visitors see
CMS (WordPress)	Hostinger	cms.qarigroup.com	Where you edit content
API	Hostinger	cms.qarigroup.com/graphql	Data bridge between the two

No ACF Pro? No problem. This guide is built entirely on free plugins. Since free ACF has no Repeater, Gallery, Flexible Content, or Options Pages, we use **Custom Post Types** for all repeatable content (projects, experience, certifications, achievements) and **textareas split by line** for simple lists (skills, bullet points). Everything stays editable without code.

Part A — Hostinger setup

A1. Create the CMS subdomain

- ☐ Log in to Hostinger **hPanel**.
- ☐ Go to **Domains** → **Subdomains**.
- ☐ Create subdomain: **cms** (full address becomes cms.qarigroup.com).
- ☐ Leave the document root at the default it suggests.

Why a separate subdomain? Keep WordPress on **cms.** and the live site on **abdul.** . This avoids cookie/CORS conflicts and lets you lock down WordPress independently.

A2. Install WordPress on the subdomain

- ☐ hPanel → **Websites** → **Auto Installer** (or "Install WordPress").

- ☐ Choose **WordPress** and select `cms.qarigroup.com` as the install location.
- ☐ Set a strong admin username (avoid `admin`) and a strong password — store it in a password manager.
- ☐ Set the admin email to your real address.
- ☐ Finish and wait for the install to complete.

A3. Enable SSL (HTTPS)

- ☐ hPanel → **Security** → **SSL**.
- ☐ Install / activate the free **Let's Encrypt SSL** for `cms.qarigroup.com` .
- ☐ Confirm `https://cms.qarigroup.com` loads with a padlock.

A4. Set PHP version & limits

- ☐ hPanel → **Advanced** → **PHP Configuration**.
- ☐ Set PHP to **8.1 or higher**.
- ☐ In PHP options, set `memory_limit` ≥ **256M**, `upload_max_filesize` ≥ **32M**, `max_execution_time` ≥ **120**.

Part B — WordPress base configuration

Log in at <https://cms.qarigroup.com/wp-admin> and do the following once.

- ☐ **Settings** → **General**: set Site Title and Tagline.
- ☐ **Settings** → **Permalinks**: select **Post name**. (Required — WPGraphQL and clean URLs depend on this.)
- ☐ **Settings** → **Reading**: tick "*Discourage search engines from indexing*" — the CMS itself should never rank; only your Next.js site should.
- ☐ **Settings** → **Discussion**: uncheck "Allow comments" (a portfolio CMS needs no comments).
- ☐ **Appearance**: leave the default theme installed but you won't use it — the front-end is Next.js.
- ☐ Delete the sample "Hello World" post and "Sample Page".

Part C — Install plugins

Go to **Plugins** → **Add New**, search each by name, install and activate. All are free.

Plugin	Purpose	Cost
WPGraphQL	Exposes your content at <code>/graphql</code>	Free
Advanced Custom Fields (ACF)	Adds custom fields to posts & pages	Free
WPGraphQL for Advanced Custom Fields	Makes ACF fields queryable in GraphQL	Free
Custom Post Type UI (CPT UI)	Create post types without code; has built-in "Show in GraphQL"	Free
Wordfence Security (or similar)	Firewall + login protection	Free

Custom Post Type UI v1.9.0+ supports WPGraphQL natively, so you do *not* need a separate bridge plugin for it.

Part D — Build the content model

This is the heart of the setup. We map each portfolio section to either a **Custom Post Type** (repeatable items) or an **ACF field group on a single Page** (one-off content).

D1. Create the Custom Post Types (CPT UI)

For each post type below: **CPT UI** → **Add/Edit Post Types**, fill the slug, then under the **GraphQL** settings enable "*Show in GraphQL*" and set the GraphQL single/plural names. Enable **Featured Image** support where noted (Supports → Featured Image).

Post type (slug)	GraphQL names	Featured image?	Holds
<code>project</code>	project / projects	Yes	Each portfolio project
<code>experience</code>	experience / experiences	No	Each job/role
<code>certification</code>	certification / certifications	Yes (the cert image)	Each certificate
<code>achievement</code>	achievement / achievements	No	Each stat card

Blog posts use WordPress's **built-in Posts** — no CPT needed (you get categories, dates and featured images for free). Make sure built-in Posts are exposed: they are by default in WPGraphQL.

D2. Add ACF fields to each Custom Post Type

ACF → **Field Groups** → **Add New**. For every group, scroll to **Settings** → **Show in GraphQL = Yes** and set a **GraphQL Field Name**. Set "Location" to the matching post type.

Group: Project Fields location: Post Type = project

- `role` — Text (e.g. "HSE Officer")
- `description` — Textarea
- Title = project name · Featured image = project photo

Group: Experience Fields location: Post Type = experience

- `company` — Text
- `location` — Text
- `startDate` — Text (or Date Picker)
- `endDate` — Text (allow "Present")
- `responsibilities` — Textarea (*one bullet per line — split on newline in Next.js*)
- Title = job title

Group: Certification Fields location: Post Type = certification

- `issuer` — Text (e.g. "NEBOSH", "OSHA")
- Title = certificate name · Featured image = certificate scan

Group: Achievement Fields location: Post Type = achievement

- `statNumber` — Text (e.g. "22%", "180+", "120")
- `statLabel` — Text (e.g. "Workers Trained")

- Title = internal name (for your reference)

D3. One-off content — the "Site Content" page

Free ACF has no Options Page, so create a normal **Page** called `Site Content` and attach ACF groups to it. In Next.js you query this single page by its slug.

Group: Hero location: Page = Site Content

- `name` — Text · `headline` — Text · `intro` — Textarea
- `profileImage` — Image · `cvFile` — File (PDF résumé)

Group: About

- `bio` — WYSIWYG / Textarea
- `coreSkills` — Textarea (*one skill per line*)

Group: Contact

- `email` · `phone` · `locationText` — Text
- `linkedin` · `otherSocial` — URL/Text

The free-ACF pattern in one line: repeatable thing → a Custom Post Type; simple repeating list → a Textarea with one item per line. That covers every section without Repeater fields.

Part E — Test the GraphQL API

- ☐ In wp-admin go to **GraphQL** → **GraphiQL IDE**.
- ☐ Run a test query and confirm you get your data back:

```
query PortfolioTest {
  projects {
    nodes { title projectFields { role description } featuredImage { node
{ sourceUrl } } }
  }
  experiences {
    nodes { title experienceFields { company location startDate endDate
responsibilities } }
  }
  certifications {
    nodes { title certificationFields { issuer } featuredImage { node { sourceUrl } } }
  }
  page(id: "site-content", idType: URI) {
    heroFields { name headline intro profileImage { node { sourceUrl } } }
  }
}
```

- ☐ Confirm the public endpoint loads: <https://cms.qarigroup.com/graphql>
- ☐ Note this URL — it goes into your Next.js `.env` as `WORDPRESS_API_URL`.

CORS: if Next.js can't fetch during build, install the **WPGraphQL CORS** helper or add an `Access-Control-Allow-Origin` header for `abdul.qarigroup.com`. Server-side fetches (build time) usually don't need it; client-side ones do.

Part F — Security hardening

A headless WordPress is a known target. Lock it down:

- ☐ Activate **Wordfence** firewall and enable login rate-limiting.
- ☐ Use a strong admin password + enable **two-factor authentication**.
- ☐ Keep WordPress core, plugins and themes **auto-updating**.
- ☐ Never expose GraphQL mutations publicly — they're off by default; keep it that way.
- ☐ In **GraphQL** → **Settings**, consider disabling public introspection once development is done.
- ☐ Delete unused themes and plugins.
- ☐ Keep "Discourage search engines" ticked so the CMS never gets indexed.

Part G — Enter your content

- ☐ Add each **Project** (title, role, description, image).
- ☐ Add each **Experience** (job title, company, location, dates, responsibilities — one per line).

- ☐ Add each **Certification** (name, issuer, upload the certificate image as featured image).
- ☐ Add each **Achievement** (stat number + label).
- ☐ Write your **Blog** posts as standard Posts with featured images.
- ☐ Fill the **Site Content** page: Hero, About + core skills, Contact details, upload CV PDF.
- ☐ Re-run the GraphQL test query to confirm all content appears.

What's next — the Next.js side

Once the CMS returns clean data, the frontend work is:

1. Add `WORDPRESS_API_URL=https://cms.qarigroup.com/graphql` to `.env.local`.
2. Fetch via `fetch` or Apollo; render one component per section (Hero, About, Experience, Achievements, Certifications, Projects, Blog, Contact).
3. Use **Static Generation + ISR** with on-demand revalidation so edits in WordPress rebuild only the changed pages.
4. Animate with **Tailwind + Framer Motion + Lenis** (add GSAP / React Three Fiber later for signature sections).
5. Deploy to Vercel, then point `abdul.qarigroup.com` at Vercel via a DNS CNAME in Hostinger.

Ask me next for the matching Next.js folder structure, the full GraphQL queries per section, and a sample ISR + revalidation webhook — I can scaffold those into your boilerplate.